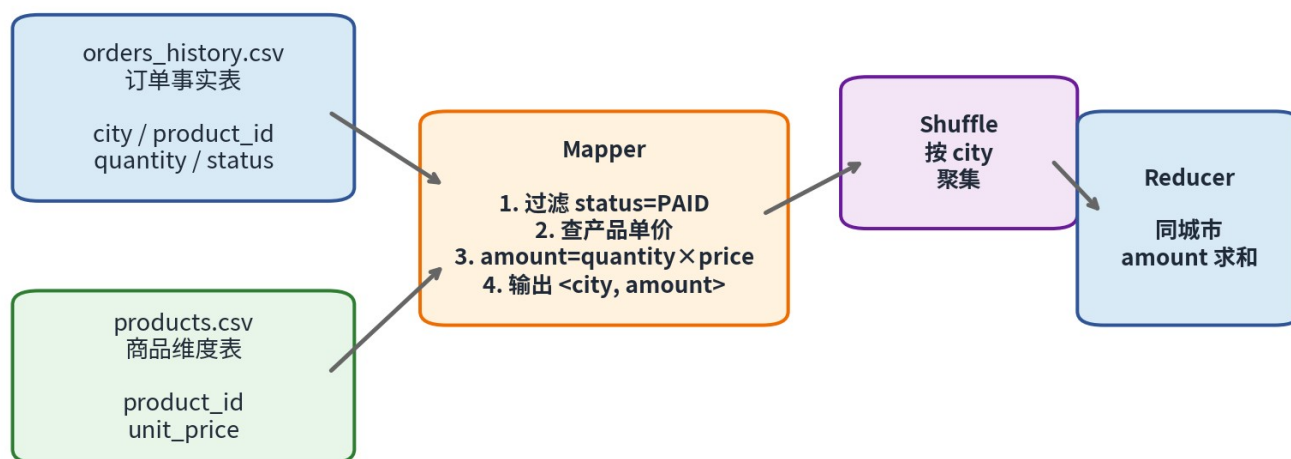


大数据综合项目实战

第3讲：用 Java MapReduce 统计各城市 PAID 订单销售额

第一次把 HDFS 数据真正送进 YARN / MapReduce 计算，并用标准答案验收

第3讲业务目标：统计各城市 PAID 订单销售额



最终标准答案：Beijing 6874 | Shanghai 6984 | Guangzhou 676 | Shenzhen 299 | 总计 14833

本讲最终目标

从 HDFS 中读取 `orders_history.csv`；用 Distributed Cache 加载 `products.csv`；Mapper 计算订单销售额并输出 `<city, amount>`；Reducer 按城市求和；最终四城结果必须与标准答案完全一致。

统一环境：Ubuntu 22.04 + Hadoop 3.3.5 + JDK 1.8；Windows 端统一使用 VS Code Remote SSH 操作 Ubuntu 虚拟机。

1. 先从业务问题开始：到底要算什么？

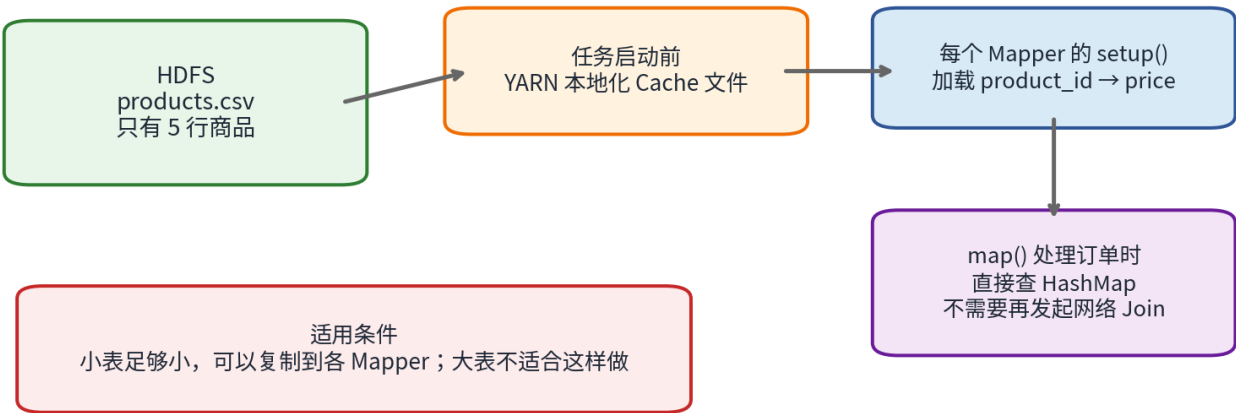
业务需求：只统计 status=PAID 的历史订单，计算每个城市的销售额。销售额的定义是 $quantity \times unit_price$ 。CANCELLED 和 REFUNDED 不计入本次销售额。

为什么不能只读 orders_history.csv?

订单表只有 product_id 和 quantity，没有 unit_price。单价存在 products.csv 中。因此，本讲必须完成一次“订单事实表 + 商品维度表”的关联。

字段	来自哪张表	业务含义	本讲用途
city	orders_history.csv	订单发生城市	Map 输出 Key
product_id	orders_history.csv	商品编号	查 products.csv
quantity	orders_history.csv	购买数量	计算金额
status	orders_history.csv	PAID/CANCELLED/REFUNDED	过滤
unit_price	products.csv	商品单价（元）	$quantity \times unit_price$

Map-side Join：为什么把 products.csv 放进 Distributed Cache？



本项目 products.csv 只有 5 行，因此非常适合 Map-side Join。

为什么选 Map-side Join?

products.csv 只有 5 行，是典型的小维度表。把它复制到每个 Mapper 本地比让两张表都进入 Shuffle 更简单、更省网络。

2. 先手算标准答案：程序还没写，结果先确定

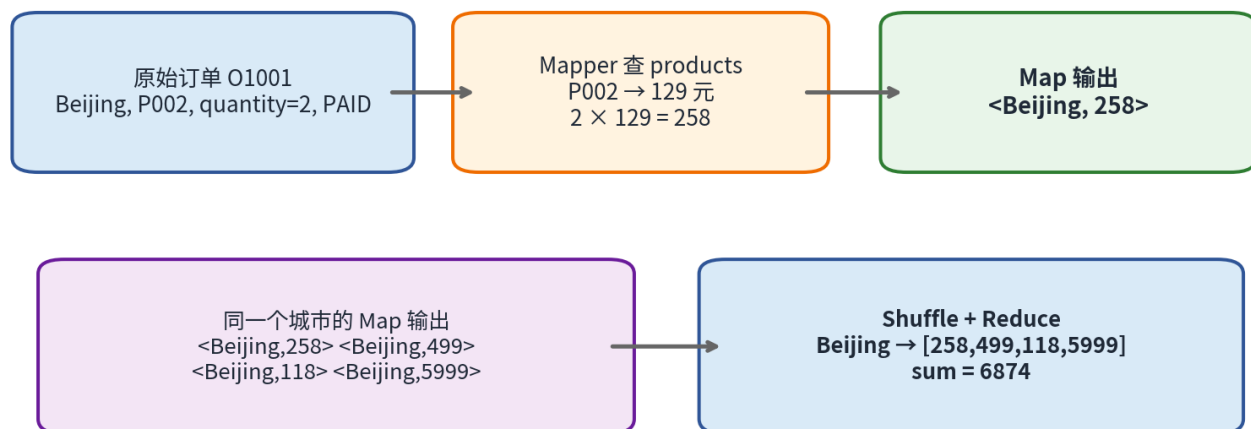
这是本项目非常重要的习惯：先建立“小数据标准答案”，再跑分布式程序。这样 Job 即使显示 SUCCESS，只要结果不等于标准答案，也不能验收。

订单	城市	商品	数量	单价	销售额
O1001	Beijing	P002	2	129	258
O1002	Shanghai	P001	1	5999	5999
O1003	Guangzhou	P003	3	59	177
O1005	Beijing	P005	1	499	499

O1007	Shanghai	P004	2	299	598
O1008	Guangzhou	P005	1	499	499
O1009	Beijing	P003	2	59	118
O1010	Beijing	P001	1	5999	5999
O1011	Shenzhen	P004	1	299	299
O1012	Shanghai	P002	3	129	387
城市			标准销售额		
Beijing			6874		
Shanghai			6984		
Guangzhou			676		
Shenzhen			299		
TOTAL			14833		

3. 把业务问题翻译成 MapReduce

从一条订单到 Reducer : Key / Value 到底怎样变化？



关键：业务问题要求“同城市见面”，所以 Map 输出 Key 设计为 city。

MapReduce 建模的核心不是先想“Java 类怎么写”，而是先问：哪些数据最后必须见面？这里同一个城市的订单金额必须见面，所以 city 就是中间 Key。

阶段	输入	输出	本讲含义
Mapper	一行订单	<city, amount>	过滤 PAID，查单价，算金额
Partition/Shuffle	所有 Map 输出	同 city 到同 Reducer	让同城市金额见面
Reducer	<city, [amount...]>	<city, total>	把同城市所有金额求和
Combiner	Map 本地的 city 金额	局部小计	安全减少 Shuffle 数据

为什么 Combiner 可以用？

本讲 Reduce 操作只是整数求和，满足结合律和交换律。局部先加再全局加，结果不变。因此可以安全复用 SalesReducer 作为 Combiner。

4. 上机前：重新启动虚拟机、VS Code 和 Hadoop 服务

仍按固定规范：每次课都假设虚拟机刚打开、HDFS/YARN 尚未启动。不要依赖“上节课可能还在运行”。

```
# 1) 远程终端确认
whoami
hostname
pwd
cd ~/bigdata-capstone

# 2) 启动前检查
jps
hadoop version

# 3) 启动 HDFS / YARN
start-dfs.sh
start-yarn.sh

# 4) 健康验证
jps
hdfs dfsadmin -report
yarn node -list
```

jps 至少应重点看到

NameNode、DataNode、ResourceManager、NodeManager（以及 Jps）。若 HDFS 或 YARN 没有健康起来，不要进入 Java 编译步骤。

5. 先验证第 2 讲的 HDFS 输入还在

```
hdfs dfs -ls -R /bigdata-capstone/raw

hdfs dfs -cat /bigdata-capstone/raw/orders/orders_history.csv
hdfs dfs -cat /bigdata-capstone/raw/products/products.csv

# 行数: orders 应为 13 行 (1 表头 + 12 数据) ; products 应为 6 行
hdfs dfs -cat /bigdata-capstone/raw/orders/orders_history.csv | wc -l
hdfs dfs -cat /bigdata-capstone/raw/products/products.csv | wc -l
```

为什么这一小步必须做？

如果输入文件缺失、路径写错或内容不对，后面任何 Java 调试都是浪费时间。分布式项目首先验证输入证据。

6. 在 VS Code 中创建第 3 讲代码目录

在左侧 Explorer 打开远程目录 /home/hadoop/bigdata-capstone。然后在远程终端执行：

```
cd ~/bigdata-capstone
mkdir -p mapreduce/lecture3/src
mkdir -p mapreduce/lecture3/build
cd mapreduce/lecture3
pwd
ls -lah
```

接着在 VS Code 左侧找到 mapreduce → lecture3 → src，创建文件 CityPaidSales.java。你也可以使用本讲代码包中的现成源文件，但建议至少完整阅读一次代码。

7. Java 程序第一部分：Mapper 为什么要先加载商品价格？

Mapper 的 setup() 每个 Mapper Task 只调用一次。非常适合加载小维度表；而 map() 会对每一条订单调用。把 products.csv 放到 setup() 加载，可以避免每处理一条订单就重新读取一次文件。

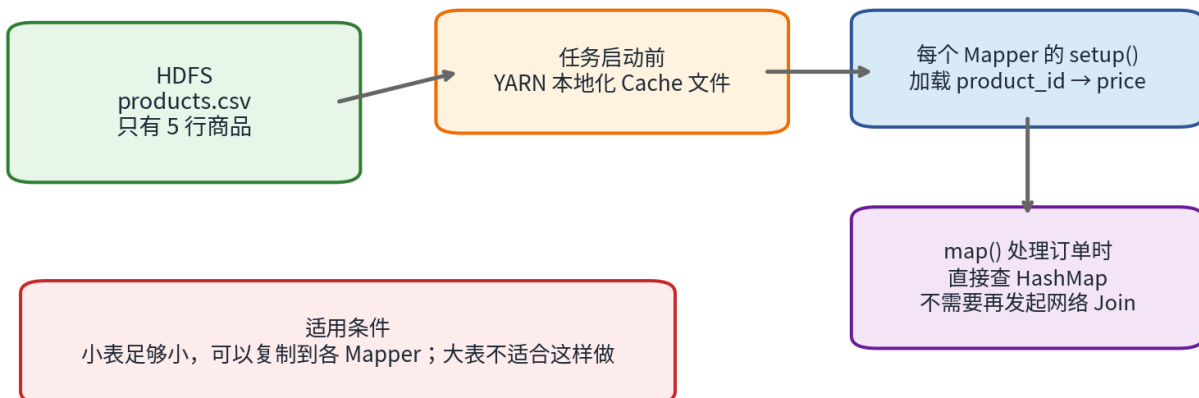
```
private final Map<String, Long> productPrice = new HashMap<>();

@Override
protected void setup(Context context) throws IOException {
    try (BufferedReader reader = new BufferedReader(new FileReader("products.csv"))) {
        String line;
        while ((line = reader.readLine()) != null) {
            if (line.startsWith("product_id,")) continue;

            String[] fields = line.split(",", -1);
            if (fields.length != 4) {
                context.getCounter(DataCounter.BAD_PRODUCT_ROWS).increment(1);
                continue;
            }

            String productId = fields[0].trim();
            long unitPrice = Long.parseLong(fields[3].trim());
            productPrice.put(productId, unitPrice);
            context.getCounter(DataCounter.PRODUCT_ROWS).increment(1);
        }
    }
}
```

Map-side Join：为什么把 products.csv 放进 Distributed Cache？



本项目 products.csv 只有 5 行，因此非常适合 Map-side Join。

8. Java 程序第二部分：Mapper 一条订单到底怎样处理？

```
String[] fields = line.split(",", -1);

String productId = fields[3].trim();
String quantityText = fields[4].trim();
String status = fields[5].trim();
String city = fields[6].trim();

if (!"PAID".equals(status)) {
    context.getCounter(DataCounter.NON_PAID_ROWS).increment(1);
    return;
}

long quantity = Long.parseLong(quantityText);
Long unitPrice = productPrice.get(productId);
long amount = quantity * unitPrice;

outCity.set(city);
outAmount.set(amount);
context.write(outCity, outAmount);
```

以 O1001 为例：city=Beijing, product_id=P002, quantity=2, status=PAID；Mapper 查到 P002 单价 129，于是产生 <Beijing,258>。

字段下标第一次必须弄清

orders_history.csv 的 7 个字段依次是：0 order_id、1 order_time、2 user_id、3 product_id、4 quantity、5 status、6 city。下标写错是初学者常见业务错误。

9. Reducer、Combiner 与 Driver

```
public static class SalesReducer
    extends Reducer<Text, LongWritable, Text, LongWritable> {

    @Override
    protected void reduce(Text city, Iterable<LongWritable> amounts, Context context)
        throws IOException, InterruptedException {
        long sum = 0L;
        for (LongWritable amount : amounts) {
            sum += amount.get();
        }
        context.write(city, new LongWritable(sum));
    }
}
```

Driver 负责告诉 Hadoop：输入在哪里、输出在哪里、Mapper/Reducer 是谁、类型是什么，以及 products.csv 需要被分发到各 Mapper。

```
job.setMapperClass(SalesMapper.class);
job.setCombinerClass(SalesReducer.class);
job.setReducerClass(SalesReducer.class);
job.setNumReduceTasks(1);

FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

job.addCacheFile(new URI(args[2] + "#products.csv"));
```

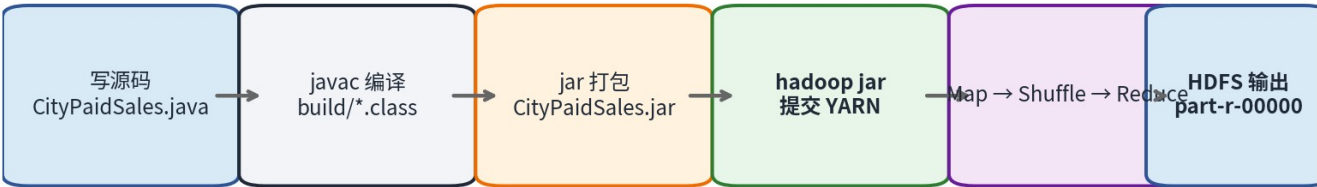
#products.csv 是什么？

URI 末尾的 fragment 给本地化后的缓存文件指定别名。Mapper 的 setup() 因而可以直接用 FileReader("products.csv") 读取它。

10. 完整源码：不要自己拼片段

完整 CityPaidSales.java 已放入本讲代码包。下面给出完整程序的结构图，实际操作时请使用代码包中的源文件，避免漏 import、漏括号或漏配置。

从 Java 源码到 HDFS 结果：完整执行闭环



验收不是“Job SUCCESS”就结束：还要核对结果、Counters、日志和标准答案。

代码区域	你应该理解什么
DataCounter	不是装饰：用于证明输入、过滤、有效订单和异常数量
setup()	每个 Mapper 启动时加载 products.csv 到 HashMap
map()	过滤 PAID → 查价格 → 算 amount → 输出 city,amount
reduce()	同城市金额求和
main() / Driver	配置 Job、I/O 类型、路径、Cache、Reducer 数量

为什么 PDF 不把 150 行源码全部挤在一页？

因为这会降低可读性。源码包中提供可直接编译的完整版本；PDF 负责逐块解释“每部分为什么存在”。

11. 保姆式编译：Java 源码 → .class → Jar

先把代码包中的 CityPaidSales.java 放到 ~/bigdata-capstone/mapreduce/lecture3/src/。如果你手工创建了文件，先保存（Ctrl+S）。

```
cd ~/bigdata-capstone/mapreduce/lecture3

# 1) 看源码是否存在
ls -lh src/CityPaidSales.java

# 2) 清空旧 build, 重新创建
rm -rf build
mkdir -p build

# 3) 编译。$(hadoop classpath) 会展开为 Hadoop 所需依赖 Jar
javac -encoding UTF-8 \
    -classpath "$(hadoop classpath)" \
    -d build \
    src/CityPaidSales.java

# 4) 看 class 文件
find build -type f -name '*.class' -print

# 5) 打 Jar
jar -cvf CityPaidSales.jar -C build .

# 6) 验证 Jar 内部
jar -tf CityPaidSales.jar | sed -n '1,30p'
```

最低构建证据

你必须看到 CityPaidSales.class、CityPaidSales\$SalesMapper.class、CityPaidSales\$SalesReducer.class 等 class 文件。只看到 Jar 文件名并不能证明它打包正确。

12. 提交 MapReduce Job：第一次真正让 YARN 计算项目数据

Hadoop 要求输出目录不能预先存在，所以每次重复实验前先删除旧输出。

```
# 1) 删除旧输出（第一次不存在也没关系）
hdfs dfs -rm -r -f /bigdata-capstone/output/mapreduce/city_paid_sales

# 2) 提交 Job
hadoop jar CityPaidSales.jar CityPaidSales \
    /bigdata-capstone/raw/orders/orders_history.csv \
    /bigdata-capstone/output/mapreduce/city_paid_sales \
    hdfs:///bigdata-capstone/raw/products/products.csv
```

命令的三个业务参数依次是：订单输入路径、结果输出目录、商品小表 Cache 路径。运行时终端会输出 Map/Reduce 百分比、Counters 和最终 Job 状态。

不要看到 map 100% / reduce 100% 就关掉终端

继续向下看 Job 是否真正 completed successfully，以及本讲自定义 Counter 是否合理。

13. 结果验收：必须同时过“系统正确”和“业务正确”

```
# 看输出文件
hdfs dfs -ls /bigdata-capstone/output/mapreduce/city_paid_sales

# 查看结果
hdfs dfs -cat /bigdata-capstone/output/mapreduce/city_paid_sales/part-r-00000

# 为了人工对比，也可以排序后显示
hdfs dfs -cat /bigdata-capstone/output/mapreduce/city_paid_sales/part-r-00000 | sort

# 再把四个城市加总
hdfs dfs -cat /bigdata-capstone/output/mapreduce/city_paid_sales/part-r-00000 \
| awk '{sum += $2} END {print "TOTAL=" sum}'
```

城市	期望结果
Beijing	6874
Guangzhou	676
Shanghai	6984
Shenzhen	299
TOTAL	14833

验收规则

城市金额和 TOTAL 必须全部一致。Job SUCCESS 但金额错误，仍然判定本讲未完成。

第3讲的五层证据链



14. Counters 与 YARN 日志：程序“可解”

本讲源码故意加入自定义 Counter。对于当前教学数据，你应该重点关注：INPUT_ROWS=13、HEADER_ROWS=1、NON_PAID_ROWS=2、VALID_PAID_ORDERS=10、PRODUCT_ROWS 通常每个 Mapper setup 会加载 5 条。

PRODUCT_ROWS 为什么可能乘以 Mapper 数？

Counter 在每个 Mapper 的 setup() 中增加。若将来输入被拆成多个 Mapper，每个 Mapper 都会各自加载 5 个商品，因此 PRODUCT_ROWS 可能是 5 × Mapper 数。这不是业务订单数量。

```
# 先查看近期 YARN Application
yarn application -list -appStates ALL

# 找到你的 application_... 后查看状态
yarn application -status application_XXXXXXXXXX_XXXX

# 查看日志（把 ID 替换成真实值）
yarn logs -applicationId application_XXXXXXXXXX_XXXX | less
```

出现失败时，在 less 中可以搜索 /Exception、/Caused by、/products.csv、/FileAlreadyExistsException。不要从几千行日志第一行开始盲看。

15. 最常见错误：按层排查，不要乱改代码

现象	优先检查	典型原因
FileAlreadyExistsException	HDFS 输出路径	旧输出目录未删除
products.csv 找不到	Distributed Cache / 参数 3	Cache URI 写错或 fragment 缺失
NumberFormatException	CSV 字段与下标	quantity/price 字段写错
结果只有部分城市	status / product / city	过滤或字段下标错误
ClassNotFoundException	Jar 内容 / 主类名	编译或打包路径错误
Connection refused	HDFS/YARN 服务	NameNode/ResourceManager 未启动

固定排错顺序

环境 → HDFS 输入 → 构建/Jar → Job 参数 → Mapper 逻辑 → Shuffle/Reducer → 输出 → 业务结果。一次只改一件事。

16. 故意制造 3 个错误：真正学会排错

建议在成功跑通后再做下面实验。每次实验都先记录“预期会发生什么”，再观察实际错误。

实验 A：不删除输出目录，直接再次运行

预期：Job 在输出检查阶段失败，常见 FileAlreadyExistsException。结论：MapReduce 输出目录要求不存在。

实验 B：把 products Cache 路径故意写成 products_missing.csv

预期：Mapper setup 无法读取 products.csv，Job 失败。结论：小表本地化也是 Job 正确性的依赖。

实验 C：把 O1001 的 status 临时从 PAID 改成 CANCELLED 再上传到一个测试路径

预期：Beijing 少 258，总额从 14833 变为 14575。结论：业务过滤条件比“程序是否报错”更重要。

17. 迁移练习：不要只会“城市销售额”这一题

题号	问题
练习 1	如果需求改成“每个城市 PAID 订单数量”，Mapper 应输出什么？
练习 2	如果改成“每个商品 PAID 销售额”，Key 应该从 city 变成什么？

练习 3	为什么 products.csv 适合 Distributed Cache，而 2 TB 的商品行为明细不适合？
练习 4	如果输出 4 个 part-r 文件，说明什么？结果一定错吗？
练习 5	如果使用平均单价，为什么不能简单把局部平均值直接相加或平均？

18. 参考答案

题号	答案
1	<city, 1>，Reducer 对 1 求和。
2	product_id。因为业务要求“同商品见面”。
3	Distributed Cache 会把小表复制到 Task 本地；大表复制到每个 Mapper 会造成巨大的网络 and 磁盘成本。
4	说明 Reducer 数量可能大于 1；只要 Partition 规则和业务聚合正确，结果不一定错，只是分散在多个 part-r 文件。
5	平均值需要同时维护 sum 与 count；“平均的平均”在分组样本数不同的时候会产生错误。

19. 本讲最终验收清单

- HDFS 与 YARN 服务健康；jps、dfsadmin、yarn node 有证据。
- HDFS 中 orders_history.csv 与 products.csv 可读。
- 能解释为什么销售额必须 Join products.csv。
- 能解释 Mapper 为什么输出 <city, amount>。
- 能解释 Distributed Cache 为什么适合 products.csv。
- Java 编译成功，Jar 中 class 完整。
- Job 在 YARN 中成功结束。
- 输出四城市金额分别为 6874 / 6984 / 676 / 299。
- 四城市 TOTAL=14833。
- 能根据 Counter 和日志判断过滤了 2 条非 PAID 订单。
- 能独立解释输出目录已存在、Cache 缺失、字段下标错误三类故障。

本讲最低闭环能力

不借助脚本时，你也能完成：确认输入 → 编译 → 打 Jar → 提交 → 查看结果 → 对标准答案 → 看 Counter/日志 → 判断错误在哪一层。

第3讲的五层证据链



20. 课后停止服务

```
# 先确认没有仍在运行的实验 Job
yarn application -list

# 停止 YARN
stop-yarn.sh

# 停止 HDFS
stop-dfs.sh

# 最后检查
jps
```

正常情况下，停止后主要只剩 Jps（以及与你环境有关的其他普通 Java 进程）。不要直接强制关虚拟机来代替正常停止服务。

21. 下一讲预告：同一业务改用 Spark DataFrame

第 4 讲将故意计算完全相同的业务问题：各城市 PAID 订单销售额。但这一次不用手写 Mapper/Reducer，而是用 Spark DataFrame 完成读取、Join、过滤、派生 amount、groupBy 聚合和 Parquet 输出。重点不是“换一种语法”，而是比较 MapReduce 与 Spark 在开发模型、Shuffle、执行计划和中间结果组织上的差别。

为什么下一讲还算同一个问题？

固定业务问题，改变计算引擎，才能真正看清 MapReduce 与 Spark 的差异，而不是被不同数据和不同需求干扰。